Building Autonomy That Cannot Lie To You

A software-in-the-loop methodology report

mYa Solutions · Jean AI 3.0 · Marine Autonomy Ayinde Rudolph, Ed.D. · arudolph@myasolutions.org Sessions across August and September 2026 Composed from two provenance records: 2026-08-19 to 08-28, and 2026-09-08 to 09-29.

What was achieved

An autonomous underwater vehicle plans and flies a complete structure inspection with no human in the loop.

It descends from its launch depth, releases thrust and measures the water's density from its own rate of rise, transits to a declared structure, acquires a standoff position and holds it against a tidal current, descends to the base of its inspection window, sweeps 8.871 m up the structure face at 98.7% of its commanded rate, refuses to go below its floor margin and records what it therefore did not inspect, returns to its start point, ascends, and holds at the surface.

Ten phases. 328 seconds. Clean exit. Every number scored against simulator ground truth rather than against the vehicle's own estimate.

vertical sweep	**8.871 m** at **0.0789 m/s** against 0.0800 commanded
tracking a *moving* depth setpoint	median error **0.075 m**, max 0.149 m
lateral position held during the sweep	within **0.348 m**
depth within ±0.15 m of setpoint	**115.3 s continuous**
decision layer active	non-zero margin on **96.0%** of 3,233 samples
refusals	fired at the floor margin, and at a position-budget expiry predicted to within **0.1 s**

The vehicle does the job. It also declines part of the job, on a stated basis, and records the decline.

What made it trustworthy

Getting there surfaced **over a hundred defects**, and that is the more useful product — not because the software was unusually bad, but because most of them would have been **invisible** on real hardware.

The sharpest example is the one that took longest to see. For the life of the project, the vehicle's decision layer was returning an identical score for every option it was asked to compare. Margin `0.0000`, on 3,235 of 3,235 samples. The mission still flew, because a regulator with a stale multiplier was doing the work — so the logs showed a reasoning system making choices it was not making.

Nothing detected it. Not the unit tests. Not a hundred defect investigations. Not ten days of flying. It surfaced only when we printed one column and the zeros went all the way down.

A system that fails loudly is a nuisance. A system that fails silently and plausibly is a liability — and underwater there is no one watching to notice. Closing that gap is what the rest of this report is about, and it is what the mission above is evidence for.

The three commitments that came out of it:

Nothing is trusted by default. Every constant carries a tag — MEASURED, DECLARED, DERIVED — naming where it came from, so a recollection cannot masquerade as a measurement. Section 5a shows what happens when that slips.

Refusal is a capability, not a failure. The vehicle declares when its position belief has expired and surfaces rather than flying a route in a frame it can no longer locate. It stops a sweep at its floor margin and writes down the part of the structure it did not see. A mission where the vehicle does everything asked is a mission with no judgement in it.

Predictions go on the record before the numbers. Five are printed above every flight. Two of them failed in the final run, and both failures are in section 4 rather than omitted from it.

1. What was built

A BlueROV2-class vehicle in a Gazebo/ROS 2 simulation of the west end of Carquinez Strait, at the USGS ADCP site characterised in Burau, Simpson & Cheng 1993 (WRI 92-4064). The 16.8 m water column comes from that deployment.

A correction to our own earlier language. This site has been described throughout the programme as a *stratified* current. It is not. The source states the deployment coincided with minimal freshwater discharge and therefore minor vertical stratification, and our own claim register recorded that a month

ago: the 50.4 to 80.1 cm/s gradient from bed to surface is bottom friction, not a salt wedge, and using the dataset as a stratified case is invalid. It is a **tidal** current, and every result below is stated that way. The word had propagated into file headers, run logs and a draft of this report before the contradiction was noticed — which is its own small instance of the pattern in section 5a.

The stack is layered with a hard boundary between reasoning and the world:

```
None
  jean_reason.py   POWELL · CFADS · RANDOLPH · BARCA · capability gate
  jean_fix.py      densitometry, drift budget, fix types
        |          NO I/O. Enforced by a pre-flight check that greps for
        |          socket and rclpy imports and REFUSES if either appears.
  jean_inspect.py  the mission: ten phases, refusals, the record
        |          UDP 4001 / 4002
  marine_shim.py   sensor synthesis and the wrench delta publisher
        |          ROS 2
  Gazebo           the plant
```

The UDP boundary is the most valuable architectural property in the system. It is what lets Gazebo be replaced by real hardware without the autonomy changing at all.

---

2. The method

Five practices, each of which caught something the others missed.

Predict before measuring

Every instrument states what it expects before it commands anything, and the prediction lands in the log above the number. `jean_inspect` prints five:

> P1 REFUSE_POS fires on the horizontal axis at 458.7 s and she abandons the route and surfaces. Correct, not a failure.

Observed: **458.8 s**. Across four runs, never outside 458.9.

P2 predicted the density sounding would return 0.1399 m/s and 1012.2 kg/m³, and recorded that this tests `c2_z` and the hull volume rather than the water — the loop closes on itself. It returned 0.105–0.119 m/s and 1006–1008 kg/m³ consistently. **P2 failed**, it is in the record as a failure, and the cause is a measurement window that includes the acceleration transient.

A prediction you can't lose is not a prediction.

Tag every constant with its basis

MEASURED, DECLARED, DERIVED, or DECLARED-ABSENT. The tag travels with the value, in the file, next to the number.

This caught **D88**, which is the most instructive defect in the record. `C2_Z = 73.225` was tagged MEASURED in four separate files. It is not measured. `model.sdf:170` declares `<zWabsW>-73.225</zWabsW>`, and `tether.py:134` says so plainly: *"declared in the model, carried unchanged."* The value agrees with the simulator to 0.0096% because it **is** the simulator's input.

And the vehicle's own SDF header knew. Line 29: the plugin coefficients are *"the independent ground truth against which our own drag measurement is checked, so they are never retyped."* Somewhere downstream the check became the measurement.

`TRIM_N` inherits it: `jean_fix.py:419` derives 1.433 N as `c2_z · v²` using the declared `c2_z`. Trim and drag are not independent measurements of anything.

Refuse rather than guess

An instrument that cannot measure says so. It does not return its fallback as a value.

The vehicle does the same thing in flight. `REFUSE_POS` fires when the drift budget is exhausted, and she abandons the route and surfaces rather than continuing to fly a path defined in a frame she can no longer locate. The sweep stops at the floor margin and the uninspected skirt is recorded as an event, in red, on the visualisation — a mission where the vehicle does everything asked is a mission with no judgement in it.

One leg per stack start

Every clean measurement came from a first leg after a fresh spawn. Every confusing one came from a body that had already been commanded.

Six "reproductions" of a directional asymmetry turned out to be the second leg of a pair hitting an already-arrested hull. We found it only by running the down leg first, which in ten days had never been done.

Score against ground truth, never against belief

The vehicle navigates on synthesised sensors with real bias and real noise. Every published number is scored against the simulator's ground truth, not against what the vehicle thinks.

Anyone can hold a setpoint their own sensor reports.

---

3. What the mission does

```
None
  0.0 ->   6.2 s   acquire          spawn depth to transit depth
  6.2 ->  21.4 s   sound            release, measure free rise, derive density
 21.4 ->  61.6 s   transit_out      9 m east under closed-loop control
 61.6 -> 116.3 s   acquire_station  settle at a 3 m standoff
116.3 -> 143.4 s   descend_scan     down to the sweep base
143.4 -> 255.9 s   scan             the inspection sweep
255.9 -> 271.1 s   depart           back to transit depth
271.1 -> 275.5 s   transit_home
275.5 -> 304.7 s   ascend
304.7 -> 328.6 s   final_hold
```

The sweep runs **upward**, for three reasons in order of weight: loss of authority during an ascent ends at the surface, while during a descent beside a structure it ends in the structure; the depth bias grows with depth, so an upward sweep carries improving positional metadata; and arriving deep leaves her at transit depth for the run home.

---

4. Measured results

All scored against simulator ground truth.

result	**value**
mission	**ten phases, clean exit, 328.4 s** of simulated time
vertical sweep	**8.871 m**, 13.851 m to 4.980 m
sweep rate	**0.0789 m/s** against 0.0800 commanded — **98.7%**
ramp tracking, on truth	median **0.075 m**, p95 0.116 m, max 0.149 m
lateral excursion during sweep	median 0.290 m, max 0.348 m

result	value
depth within ±0.15 m of setpoint	**115.3 s continuous**, on truth
depth within ±0.50 m	**302.9 s continuous**
position error, belief vs truth	max over path **0.485 m**
depth belief vs truth	mean +0.075 m, σ 0.060 m
decision layer live	non-zero margin on **96.0%** of 3,233 samples
seabed proven	dropped probe rests at **16.750 m** against 16.75 predicted
position-void refusal	**458.8 s** against 458.7 s predicted, earlier run

Two qualifications, both in the record rather than omitted from it.

The decision margins are small. Median 0.0162 N against a decisiveness threshold of 0.20 N, so only 0.9% of individual decisions clear it. The *distribution* of winners is physically sensible — `CorrectHard` during transit when the gap is large, `Hold` on the horizontal during depart when she is on station — but the per-tick confidence is low, and a damping rule carries the indecisive ones. That is a real limitation of the value model at this scale, not a rounding note.

Position error does not cancel on the out-and-back. Prediction P3 expected terminal error to be much smaller than the maximum over the path. Both are 0.485 m, and the maximum occurs at the end. P3 failed.

---

5. The findings that mattered

D79 — the seabed that never existed

The world included a Fuel model named "sand heightmap." Its SDF declares a **COLLADA mesh**, not a heightmap. dartsim cannot build mesh collisions from SDF, and said so at every single launch since the world was created:

```
None
[Dbg] [SDFFeatures.cc:332] Mesh construction from an SDF has not been
      implemented yet for dartsim.
[Dbg] [SDFFeatures.cc:864] The geometry element of collision [ground]
      couldn't be created
```

Both at debug level. Nothing surfaced them. **The world had a visual seabed and no collision geometry for its entire existence.**

Downstream consequences, all live the whole time: `descent.plan()` refused targets near the bed to protect against a bottom that was not there; the mission's floor margin guarded nothing; a terrain-characterisation exercise measured an asset physics never loaded.

The world file also carried its own warning, written in September: *"heightmap's own relief is unverified; check after first spawn."* Correct, in the right place, and prose. Nothing made it mandatory.

Rule 35: a warning that does not refuse is a comment.

The fix was a primitive box whose top surface sits at the declared depth, and a gate that proves it — a dropped probe coming to rest at 16.750 m against a predicted 16.75. The stack's previous check read the heightmap's *pose*, which exists whether or not the collision under it ever built.

D63 — a true number about the wrong quantity

`BED_DEPTH_M` was 14.7 m in one file and 16.8 m in another. The comment on 14.7 read *"deepest ADCP bin, WRI 92-4064"* — and it was true. The ADCP is bottom-mounted and blanks the 2.1 m directly above itself:

```
None
16.8  water column
-2.1  blanking, lowest usable bin starts here
------
14.7  deepest MEASUREMENT
```

Somebody took the deepest depth the instrument could measure and used it as the depth of the bed. Not carelessness — the most respectable kind of error, trusting an instrument's range as the extent of the world.

Rule 33: a measurement's deepest sample is not the bottom. Blanking distance, minimum range and first-bin offset are properties of the instrument, not of the site.

D109 — three branches computing the same thing

The value function scores three options — hold, correct gently, correct hard — in newtons, as *force avoided later minus force spent now*. Its loop read:

```python
for name, scale in (("Hold", 1.0), ("CorrectSoft", 1.0),
                    ("CorrectHard", 1.0)):
```

All three scales `1.0`. Every option computed an identical spend, an identical closing speed, an identical projected deviation, an identical everything. The three branches were the same calculation three times.

So the margin between the best and second-best option came out **exactly 0.0000**, on 3,235 of 3,235 samples, in every phase of every run. Hold won on a stable sort, every time.

The vehicle still flew — a driver-side patch was holding the last non-zero scale, so a PID with a stale multiplier did the work while the log reported a decision layer that was contributing nothing.

After the fix, the layer discriminates: `CorrectSoft` at 0.3 m of deviation, `CorrectHard` at 1.0 m, `Hold` at 10 m where either correction saturates. In flight, all three winners appear across the phases, and during the sweep the horizontal axis holds while depth corrects — which is precisely what a scan is. Nobody encoded that phase behaviour; it fell out of the comparison.

The pattern underneath

Nine of ten faults this period were in the **driver**, and all nine had the same shape: a correct function handed the wrong quantity.

defect	**the quantity**
D95	time since a correction *fired*, not since the error persisted
D96	a station-keeping horizon used to score a travel leg
D97	a saturated demand where a steady-state requirement belonged
D104	the same, from an adapting estimator
D105	a decisiveness flag computed correctly and discarded

defect	the quantity
D107	the D105 fix, which could hold a zero and became a stall
D111	a bind address and a destination that happened to share a value

The formula file's own header warns about exactly this:

> *The formulas are agnostic about units. That is deliberate — they are a structure for comparison. But a comparison needs a common currency and the integration must supply it.*

It earned that warning four times over in three weeks.

5a. The same defect, twice, a month apart

Two provenance records cover this programme. Reading them together shows something neither shows alone: **the failures recur in families**, and the second instance is never recognised as the first.

The option-scale table has been wrong twice

D-6, 2026-08-20. The decision layer's `SCALE` table read `{Hold: 0.60, CorrectSoft: 0.45, CorrectHard: 1.00}`. The option named "correct gently" commanded **less** authority than the option named "hold." `CorrectSoft` won 43,860 of 64,514 depth samples, so the loop ran at 45% authority for most of a run: 0.350 m median error against 0.035 m from the fixed policy an hour earlier.

It survived because a second defect was hiding it. A wall-clock integral (D-2) had inflated effective `KI` roughly tenfold, compensating for the throttle. **Two defects concealing each other; neither visible while both were present.** Fixing the clock exposed the scale.

D109, 2026-09-29. The scoring function's *own* option scales — a different table, same purpose, same file — read `1.0` for all three. Every option computed an identical spend, closing speed and projected deviation. Margin came out exactly `0.0000` on 3,235 of 3,235 samples, and `Hold` won on a stable sort in every phase of every run for the life of the project.

It survived because a driver-side patch was holding the last non-zero scale, so a regulator with a stale multiplier did the work while the log reported a decision layer contributing nothing. **Again two mechanisms, one masking the other.**

Same table, same function, same class of error, thirty-nine days apart, caught by different accidents both times.

The environment assumed present, never verified

D-12, 2026-08-21. Phase 2B and Phase 3 both passed and reported station-keeping results. Both had run **with the current rig inactive** — in still water. Found by asking whether the rig had been running, then testing the traces for a directional offset: 0.0038 m of mean displacement against a current that carries the hull 10.6 m per minute unopposed.

The rule already existed. Phase 0 had established that no control sits above an interface not shown to produce its intended physical effect. It had been applied to actuation and never to the disturbance.

D79, 2026-09-17. The world had **no seabed collision** for its entire existence. A mesh the physics engine could not build, announced at debug level at every launch. The stack's seabed check read the model's *pose*, which exists whether or not the collision under it ever built.

Both are the same question unasked: *is the thing I am assuming is there, there?* D-12 earned R-10. D79 earned R-35. The second was written without recognising it as the first.

A coefficient that describes something other than the hull

D-17, 2026-08-28. Two successive surge-drag fits were measured with the current rig running, fitting applied force against **ground** speed. The simulator computes drag from velocity relative to the water, so the coefficients absorbed the current instead of describing the hull. The error was 10× at 5,000 N and 25× at 500 N, and it invalidated every force and energy figure in four downstream analyses.

Found by a single wrench published by hand, no harness involved.

D88, 2026-09-17. `C2_Z = 73.225` is tagged MEASURED in four files. It is the simulator's own input, read from the vehicle model — which is why it agrees to 0.0096%. The model's header knew: the plugin coefficients are *"the independent ground truth against which our own drag measurement is checked, so they are never retyped."* The check became the measurement.

D-17 is a measurement taken in the wrong conditions. D88 is a measurement not taken at all. Both end with a number that describes the model rather than the vehicle, carrying a MEASURED tag.

And it is still happening

C-13, 2026-08-28, records BlueROV2 usable thrust as `33.732 × 1.5² = 75.897 N`, tagged **DERIVED, WEAK**, with the note that the 1.5 m/s maximum speed was *"asserted from recollection, not*

retrieved from a manufacturer document." It names its own falsification: a terminal-velocity sweep of the model, which has never been run.

`jean_reason.py` line 119 reads:

```python
AUTH_XY_N = 76.0           # MEASURED per axis
```

Same number. Every authority gate, every ABORT verdict, every "% of authority" figure in the mission pre-flight, and the whole transit-cap analysis of 2026-09-20 inherits a value its own provenance record calls the weakest in the session.

Logged as D112. The record predicted this failure a month before it occurred, in a file describing the same constant.

What the pattern says

A defect ledger prevents repetition only if it is **read at the moment of writing a constant**, not filed and consulted afterward. Ours was written with care and consulted rarely. Three of the most expensive findings of September have August ancestors that would have named them.

The corrective is not more discipline. It is making the check mechanical: a constant's tag should be derivable from its source rather than typed by whoever moved it.

---

6. Working rules

Each was written the day a specific failure earned it.

R33 A measurement's deepest sample is not the bottom.

R34 A diagnostic must report the CONDITION it measured, not select a cause from a list. Naming a mechanism is the caller's job.

R35 A warning that does not refuse is a comment.

R36 A force measurement without a same-session zero-force control measures the session, not the force.

R37 A teardown that leaves the plant in motion makes the next run's initial condition a function of the last run's exit.

R38 A control watched for a shorter time than the subject is not a control.

R39 A test that does not verify its own starting condition measures whatever state the last one left behind.

R40 Never fly straight after a whole-file deployment. Check the constants first.

R41 A fix that constrains a controller must be able to damp it, never to cancel it.

R42 Direction is geometry. A travel command's heading comes from the vector to the target, never from the sign of a regulator output.

Inherited from the August sessions

R-1 Bound every loop on simulated time, never the wall clock. *Bought with two days of runs across four gates, and a headline of 363.7 s that became ~93 s and fell below its own criterion.*

R-2 Log both clocks in every trace. *The sim-time stamp was in every odometry message from the start and no harness recorded it.*

R-3 Integrate control terms on the clock the vehicle experiences. *The published gains had never described the running loop; effective KI varied with machine speed by an order of magnitude.*

R-4 Any metric that can exceed its own physical ceiling must check itself. *A 563.1 s hold was reported inside a 60 s run, and the gate passed.*

R-5 A parse check is not an execution check.

R-6 A patch that cannot find its anchor must abort loudly, not report success.

R-7 Publish a persistent wrench only when the command changes. *6.5 hours of wall clock, zero samples. The rule already existed in two earlier harnesses and was not carried into a third.*

R-8 Long or verbose runs go through plain redirection, never a pty.

R-9 Verify the vehicle's state before and after any measurement, and refuse to compare a reading taken elsewhere.

R-10 The verification rule applies to the disturbance, not only the actuation.

R-11 One process on the simulator at a time.

R-12 State the conditions a measurement is valid under at the moment it is taken. A curve fitted in a moving fluid does not describe the hull.

R-40 is R-6 rediscovered. R-39 is R-9 rediscovered. R-36 is R-10 rediscovered. Three of the ten rules written in September already existed in August under different numbers.

R34 is the one that cost most to learn. Five separate instruments returned confident, internally consistent, **wrong** diagnoses against a single fault — each naming a mechanism from a list written before that fault existed. One reported "actuation dead" while the command path showed every message delivered. Another printed "produced no motion" directly above its own measurement of 2.06 metres of motion.

Writing the rule did not confer immunity to it. The instrument built to enforce R34 broke it on its first run.

---

7. What this cost, and what it bought

Three weeks. Over a hundred defect entries with dates and evidence.

What it bought is a stack where **every number has a provenance**, every refusal has a stated basis, and the failures that remain are named rather than lurking. The open ones are in the record with the same discipline as the closed ones:

- the sounding reads consistently low, and the cause is a measurement window that includes the acceleration transient
- a 0.19 m depth offset that is an integrator limitation
- decision margins that are small relative to the actuator's own deadband
- `C2_Z` and `TRIM_N` are DECLARED, not MEASURED, and the independent measurement has not been done

None of those is hidden. That is the point.

---

8. Why this matters for the hardware

Underwater there is no ground truth channel. No `Scorer`, no way to ask the simulator where the vehicle really is. Every defect in this record would have been **invisible** on real hardware — and several of them would have looked exactly like bad tuning, which is the diagnosis that wastes the most time and the most vehicles.

A seabed that does not exist. A drag coefficient echoed from the model it was meant to validate. A decision layer that scored every option identically for the life of the project. None of those announce themselves. All of them change what the vehicle does.

The method is the product. The mission is the demonstration that the method works.

And the same discipline applied to the next step

Before moving the autonomy onto the vehicle's own computer, we characterised the control path. The budget is derived, not guessed: the loop runs at 10 Hz, so a tick is 100 ms, and that must cover sensor transit, decision, command transit and the plant step.

Measured over a wide-area link:

```
None
median  72.61 ms    73% of the budget
p99    152.29 ms   152% of the budget
loss     1.00%
```

The loop cannot close on that path. Commands would arrive after the state they were computed from had been superseded — and that failure presents as a badly tuned controller, which is the diagnosis this programme has already learned to distrust.

Stated before any hardware was bought, from a fifty-second measurement. It is also, incidentally, the local-first thesis measured rather than asserted: autonomy has to live with the vehicle.

Appendix — reproducing this

Every result above comes from a mission record: one JSONL file, one line per control tick, written incrementally so a run that dies at minute 140 of 180 leaves 140 minutes of usable data. Each record carries its own header with the constants, the geometry and the predictions, so a reading of the data cannot drift from the conditions that produced it.

The visualisation embeds its own data — no network, no CDN, no server — and stamps a banner across any output generated from a synthetic fixture that cannot be switched off. A fixture must never be mistakable for a run.